# SVM - Support Vector Machine

## Introduction

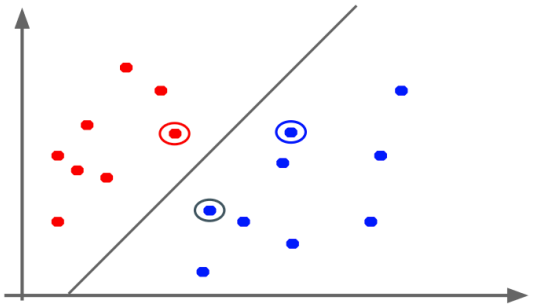
The "**advanced** version" of a **linear classifier**

- **Considers** the **ideal hyperplane** → **maximizes** the **margin**
- Can **cope with outliers**

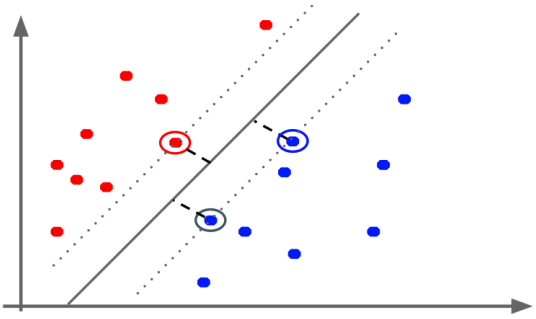
**Given** any **separating hyperplane**, the **closest points** are called the **support vectors**

- For $n$ **dimensions** there will be **at least** $n + 1$ **support vectors**

→ **To maximize the margin** only **support vectors matter**



Hyperplane and support vectors



Support vectors and margin

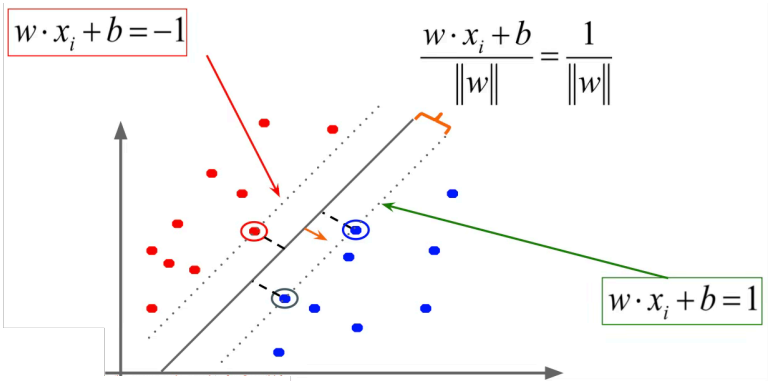
## Measuring the margin

The **line perpendicular to the hyperplane** and **passing through** the **sample** $x_i$ is defined as

$$w \cdot x_i + b = y_i$$

**If** $x_i$ is a **support vector**, $y_i$ **is** equal to $-1$ or $+1$

$$w \cdot x_i + b = \pm 1$$




Given the support vector $x_i$, **the margin length is**:

$$\frac{w \cdot x_i + b}{\|w\|} = \frac{1}{\|w\|}$$

### Maximizing the margin

We **aim** to **select the hyperplane with the largest margin**,

where **points are correctly classified** and outside the margin

It's **a constrained optimization problem**:

$$\max_{w,b} \text{margin}(w,b) = \max_{w,b} \frac{1}{\|w\|} \quad \text{subject to } y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

Moreover, maximizing $\frac{1}{\|w\|}$ is equal to minimize $\|w\|$:

$$\min_{w,b} \|w\| \quad \text{subject to } y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

Moreover, minimizing $\|w\|$ is equal to minimize $\|w\|^2$:

$$\min_{w,b} \|w\|^2 \quad \text{subject to } y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

💡 We stick at $\|w\|^2$ because it is a convex function which is easily differentiable

### Soft margin classification

With a **strict categorical constraint all examples must stay**:

- **Out** of the **margin area**
- At the **correct side of** the **margin**

We want to **cope** with **outliers** points and **not-separable** datasets

- Using **slack variables** → allow for **violations** in constraints
- Applying the **kernel trick**

$$\min_{w,b} \|w\|^2 + C \sum_i \zeta_i \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 - \zeta_i \quad \forall i, \zeta_i \geq 0$$

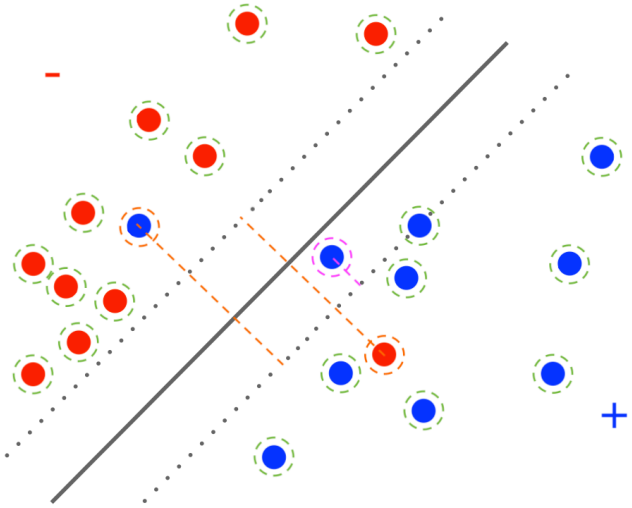
Where:

- $w$ is the **margin**
- $C$ is an **hyperparameter**, indicates the **trade-off between margin maximization and penalization**
  - **Small** $C$: **constraints** can be **easily ignored** → **large margin**
  - **High** $C$: **constraints** are **hard to ignore** → **narrow margin**
  - $C = \infty$: **enforces** all **constraints** → **hard margin**
- $\zeta_i$ **penalizes** by **how far from its class space an outlier is located**

### Slack penalty

When dealing with the slack variable:

- The **sample** is **on or beyond the margin**
  - **No** need for **correction**
  - $\zeta_i = 0$
- The **sample** is **over the margin** but on the **correct side** of the **hyperplane**
  - **Difference** from the point to the margin
  - $\zeta_i = 1 - y_i(w \cdot x_i + b)$
- The **sample** is an **outlier**
  - **Distance from** the **hyperplane plus** the **distance from** the **margin**
  - $\zeta_i = y_i(w \cdot x_i + b) - 1$




Case 1), Case 2), Case 3)

We should **define the slack variable** as following:

$$\zeta_i = \begin{cases} 0 & \text{if } y_i(w \cdot x_i + b) \geq 1 \\ 1 - y_i(w \cdot x_i + b) & \text{else} \end{cases}$$

Which is **equal** to **calculate the hinge loss function**:

$$\begin{aligned} \zeta_i &= \max(0, 1 - y_i(w \cdot x_i + b)) \\ &= \max(0, 1 - y_i y_i') \end{aligned}$$

It results to be an **unconstrained problem**

## Solving as a dual problem

We can identify 3 classes of optimization problems:

- Linear programming problem: linear problem, linear constraints
- **Quadratic programming problem**: quadratic problem, linear constraints, can be convex
- Non-linear programming problem: in general isn't convex

The previous is a **quadratic optimization problem**

### Lagrange multiplier

We want to **construct a dual problem** where **a Lagrange multiplier** $\alpha_i$ is **associated** with **every inequality constraint** in the **primal problem**

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$\text{subject to} \quad \sum_i \alpha_i y_i = 0 \; \alpha_i \geq 0, \forall i \qquad \text{with hard margin}$$

$$\text{subject to} \quad \sum_i \alpha_i y_i = 0 \; \alpha_i \geq 0, \forall i \qquad \text{with soft margin}$$

Given a solution $\alpha_1 \dots \alpha_n$ to the dual problem, **the solution to the primal is**:

$$w = \sum_i \alpha_i y_i x_i$$

$$b = y_k - \sum_i \alpha_i y_i x_i^\top x$$

The main findings are:

- **Each non-zero** $\alpha_i$ indicates that $x_i$ **is a support vector**
- **We don't need to explicitly compute the weights vector** $w$
  - $w$ can be expressed as a linear combination of support vectors

The classifying function is:

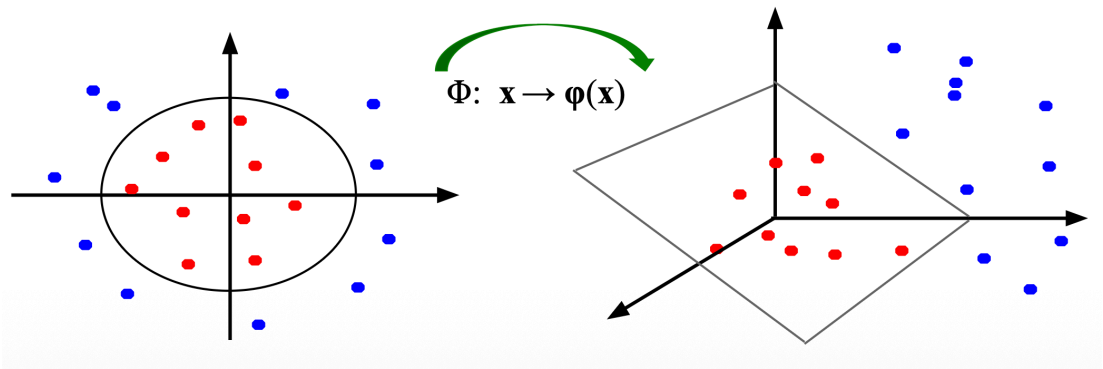
$$f(x) = \sum_i \alpha_i y_i x_i^\top x + b$$

The **solution** relies only on the **inner products** $x_i^\top x$

- Between the **test point** $x$ and the **support vector** $x_i$
- → We only **need** to **compute** and minimize the **inner products between all training points**

### Solving non-linear SVM

If it is **non-linearly separable**, the **original feature space** can **always be mapped to** some **higher-dimensional feature space where** the **training set** is **separable**




Mapping from a 2D features space to a 3D features space

#### Kernel trick

- The **linear classifier relies** on the **inner product between vectors**: $K(x_i, x_j) = x_i^\top x_j$
- **Every datapoint** can be **mapped** into a **higher-dimensional space**
  - Where datapoints are separable
  - Via some **transformation**: $\Phi : x \rightarrow \phi(x)$
  - The **inner product** in that dimension **becomes**: $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$

⚠️ **Applying** the transformation $\phi$ **wouldn't be feasible** (because of complexity)

- The **algorithm** only **computes** the **inner products** in that dimension using the kernel function $K$

### Mercer's theorem

**Defines** whether a **kernel function can be used in SVM** to **compute** the **kernel trick**.

A kernel function $K(x_i, x_j)$ is **suitable** to SVM **if and only if** it is:

- **Symmetric**: $K(x_i, x_j) = K(x_j, x_i)$
- **Positive semidefinite**:
  - The generated Gram matrix must be symmetric and positive semidefinite
  - All eigenvalues are non-negatives